

Percentile-Based Grouping in SCF Analysis

Introducing the Percentile Cut Function

Joseph N. Cohen

May 27, 2026

ABSTRACT

This note examines the task of summarizing statistics by percentile-defined groups in Survey of Consumer Finances (SCF) microdata. Percentile-based grouping is a commonplace analytical operation, but its use in SCF analysis is complicated by the data's complex design. This note reviews three methods for making such comparisons: a naive threshold approach, implicate-wise estimation using Rubin's Rules, or the Federal Reserve's official "stacking" method. The function `scf::scf_pctile_sum()` offers a structured and transparent means for comparisons by percentile-based grouping variables in SCF analysis. The note demonstrates the function by comparing top-decile mean net worth estimates across SCF survey years to Federal Reserve published estimates and to the Federal Reserve-provided `nwcat` grouping variable.

1 Introduction

This technical note discusses the task of summarizing statistics by percentile groups in analyses of the *Survey of Consumer Finances* (SCF) public-use microdata (U.S. Federal Reserve Bank 2023b). Examples of such an operation might be discerning health insurance coverage rates in the bottom 25% of household income, or the mean financial investments among those in the top 1% of net worth. The Survey's complex design and use of multiple imputation to handle missing data complicate this task. An analyst can make different choices about how percentile-group membership is operationalized and estimated, each of which produces similar but not identical results.

Below, I describe three methods for calculating statistics across percentile-defined groups in SCF data: a “commonsense” approach, the Federal Reserve’s official “stacking” method, and the implicate-wise method suggested by Rubin (1987). In this exercise, the three methods render substantively similar results. The commonsense approach renders slightly imprecise decile groupings, and Rubin’s Rules will render slightly different results than the stacking method.

In prior iterations of the `scf` package, users had to write their own code to implement percentile-based grouping procedures that respected the SCF’s multiple-imputation and weighting structure (Cohen 2026b). Here, we introduce the new function `scf_pctile_sum()`, which automates these operations. The function allows users to choose between an implicate-wise workflow for ordinary analysis and a stacked workflow for reproducing the Federal Reserve’s percentile-category convention.

2 Background

A Federal Reserve-published chartbook estimates that, in 2022, the mean net worth among the top 10% was \$7,771,300 (rounded to nearest hundreds) (U.S. Federal Reserve Bank 2023a). In 1989, this figure was \$2,926,600¹. Both figures represent statistical estimates based on percentile groupings.² How to reproduce this finding?

The SCF is a complex survey that uses multiple imputation to deal with missing values. As such, its wrangling and analysis requires an array of functions that correctly handle this nonstandard data (see Cohen 2026a). Standard R functions that do not account for the data’s complicated structure and analytical demands will produce incorrect estimates. The R package `scf` provides a comprehensive set of tools that correctly handle data.

2.1 Commonsense Approach

A commonsense strategy for producing estimates is through three steps: (1) estimate the 90th percentile values of net worth for the pop-

¹in inflation-adjusted 2022 dollars

²Note that these cited figures were produced with the SCF’s publicly-available micro-data, a privacy-conscious version of its internal-use data. As such, they may differ slightly and non-substantively from official estimates derived from the internal set.

ulation, (2) identify households in the top decile if their net worth surpasses that threshold, and (3) estimate the mean net worth of the so-identified top decile group. The chunk below exemplifies that approach:

```

1 # Load the data
2 scf2022 <- scf_load(2022)
3
4 # scf_percentile() to discern percentile values:
5 temp_90p <- scf_percentile(scf2022, ~networth, q =
6   0.9)
7
8 # The `scf` functions return lists with multiple
9   points of information.
10 # The final results are stored under the list item
11   `results`:
12 threshold <- temp_90p$results$estimate
13
14 # Create a variable demarcating a household as having
15   a
16   # net worth that surpasses threshold
17 scf2022 <- scf_update(scf2022, P90_networth =
18   networth > threshold)
19
20 # Calculate mean by group
21 scf_mean(scf2022, ~networth, ~P90_networth)

```

```

1 ## Multiply-Imputed, Replicate-Weighted Mean Estimate
2 ##
3 ##   group variable  estimate          se      min
4 ##   FALSE networth 312092.7    7138.116  306631
5 ##   TRUE  networth 7733378.0   339699.606 7335566
6 ##   7940556.7

```

This gives us a result of \$7,733,378. This is close to the rounded official estimate of \$7,771,300, but it does not replicate it. This failure to replicate is the product of two issues. First, the commonsense method is not best practice for estimating parameters across percentile-defined groups when using multiply imputed data. Second, the method does not match SCF protocol for estimating percentile scores, which slightly diverges from the approach recom-

mended in canonical missing data literature (Little and Rubin 1987; Rubin 1987).

2.2 Within-Implicate Method

One issue is that we used our global estimate of the 90th percentile net worth value to set our cutoffs. This is our estimate of the population-level 90th percentile net worth value after having processed each of the imputed sets and combined them to generate a global estimate. Each individual implicate has its own 90th percentile estimate, which you can see in the object created by the function `scf_percentile()`:

```
1 summary(temp_90p)
```

```
1 ## Summary of SCF Percentile Estimate
2 ##
3 ## Pooled Estimates:
4 ##   variable quantile   estimate      se      min
5 ##   networth      0.90 1920758.00 137527.18 1809170.00
6 ##   2079000.00
7 ##
8 ## Implicate-Level Estimates:
9 ##   implicate group quantile   estimate      se
10 ##          1   All      0.90 1877000.00 73435.75
11 ##          2   All      0.90 2079000.00 68754.72
12 ##          3   All      0.90 1866120.00 70292.91
13 ##          4   All      0.90 1972500.00 62072.35
14 ##          5   All      0.90 1809170.00 83208.51
```

The population-level 90th percentile estimate ultimately settled on \$1,920,758, but that cutoff will not get you the top 10% of each implicate. For example, implicate #1 has an estimated 90th percentile wealth value of \$1,877,000, so you will exclude observations that are above that number but not greater than the global estimate. The differences are extremely narrow, such that between 9.74% and 10.81% of individual implicates are scored as “top 10%” in our estimates using this approach. It will likely not lead to substantive differences, but will lead to estimates that do not fully replicate those generated by best practice.

2.3 Calculating Percentiles by Implicate

A standard imputation-aware procedure is to perform the complete-data analysis separately on each implicate and then pool the resulting

estimates. So we are supposed to calculate the mean net worth for the top 10 percent of each implicate separately, and then pool those five estimates using Rubin's (1987) rules. The threshold is treated as an intermediate quantity internal to each implicate's analysis, not as a single threshold value applied to each individual implicate. The earlier versions ($\leq 1.0.6$) of the `scf` package have the function `scf_update_by_implicate()` which allows users to generate variables on an implicate-by-implicate basis, such that we can create a variable that properly demarcates the top 10% scores for each implicate.

```

1 # Function to demarcate the top 10% scores within each
  implicate
2 scf2022 <- scf_update_by_implicate(scf2022,
  function(df) {
3
4   # Create array of replicate weight column names
5   rep_cols <- grep("^wt1b", names(df), value = TRUE)
6
7   # Create an implicate-specific survey design object
8   design <- survey::svrepdesign(
9     weights = ~wgt,
10    repweights = as.matrix(df[, rep_cols]),
11    data = df,
12    type = "other",
13    scale = 1,
14    rscals = rep(1 / (length(rep_cols) - 1),
15               length(rep_cols)),
16    mse = TRUE,
17    combined.weights = TRUE
18  )
19
20  # User `survey::svyquantile()` to calculate the
    90th percentile
21  # value for this implicate, and capture its value as
    a numeric object
22  p90 <- as.numeric(coef(
23    survey::svyquantile(~networth, design, quantiles
24    = 0.9, se = FALSE)
25  ))
26
27  # Create a variable that demarcates the top 10% of
    this implicate
28  df$top10nw <- df$networth >= p90
29  df
30 })
31
32 # scf_mean will apply to each implicate with the
    correctly-identified
33 # top decile
34 scf_mean(scf2022, ~networth, ~top10nw)

```

```

1 ## Multiply-Imputed, Replicate-Weighted Mean Estimate
2 ##
3 ## group variable estimate se min
  max

```

```
4 ## FALSE networth 313585.3 13551.17 302993.7  
  332144.9  
5 ## TRUE networth 7766387.1 210993.20 7680548.0  
  7852333.2
```

The implicate-wise method results in an estimate of \$7,766,387. This is close to, but distinct from, the published estimate because it targets a different workflow than the Federal Reserve’s stacked percentile-category convention. The reason is that the SCF’s official estimates are not generated by calculating the 90th percentile value for each implicate and then pooling those values. Instead of using Rubin’s Rules, the SCF’s macro stacks all five implicates, divides each row’s weight by five, sorts the stacked file by the target variable and household identifier, computes the cumulative weighted population share, and assigns percentile categories directly from that cumulative share. Published percentile-group summaries are then calculated over these assigned categories. This procedure produces estimates that are substantively very similar to those generated by the implicate-wise workflow.

3 Introducing `scf_pctile_sum()`

The above operations require functional programming. The function `scf_pctile_sum()` automates percentile-based grouping in SCF data. With `method = "implicate"`, it computes survey-weighted quantile thresholds within each implicate using `svyquantile()` from Lumley’s survey package (Lumley et al. 2026), classifies households within implicates, and pools summary estimates using the package’s multiple-imputation workflow. With `method = "stack"`, it follows the Federal Reserve convention: the five implicates are stacked, weights are divided by five, observations are sorted by the percentile variable and household identifier, percentile groups are assigned from the cumulative weighted population share, and flat weighted statistics are computed on the stacked data.

```

1 scf2022 <- scf_load(2022)
2
3 scf_pctile_sum(scf2022, ~networth,
4               probs = c(0, 0.9, 1),
5               labels = c("bottom90", "top10"),
6               method = "implicate",
7               stat = "mean",
8               stat_var = ~networth)

```

```

1 ## Multiply-Imputed, Replicate-Weighted Mean Estimate
2 ##
3 ##      group variable  estimate      se      min
4 ##      bottom90 networth 313585.3 13551.17 302993.7
5 ##      top10 networth 7766387.1 210993.20 7680548.0
6 ##      7852333.2

```

The resulting \$7,766,387 replicates the estimate using the implicate-wise method (above), but not the official estimate. As noted, these do not replicate due to differences between how percentile values are estimated by the SCF. To reproduce official estimates, use the “stacked” method:

```

1 scf_pctile_sum(scf2022, ~networth,
2               probs = c(0, 0.9, 1),
3               labels = c("bottom90", "top10"),
4               method = "stack",
5               stat = "mean",
6               stat_var = ~networth)

```

```

1 ##      group variable  estimate
2 ## 1 bottom90 networth 313517.2
3 ## 2  top10 networth 7771163.1

```

The result is approximately \$7,771,160, or \$7,771.16K in the table below. This is within about \$0.13K of the published value. I repeat the exercise with the 1989 data, where the estimate matches the published value to the displayed precision used here:

```

1 scf1989 <- scf_load(1989)
2
3 scf_pctile_sum(scf1989, ~networth,
4               probs    = c(0, 0.9, 1),
5               labels    = c("bottom90", "top10"),
6               method    = "stack",
7               stat      = "mean",
8               stat_var  = ~networth)

```

```

1 ##      group variable estimate
2 ## 1 bottom90 networth 159372.9
3 ## 2   top10 networth 2926547.5

```

The result is \$2,926,547.50, or \$2,926.55K, matching the published value to the displayed precision used here. Table 1 compares the `scf_pctile_sum()` stacked estimates to published Federal Reserve values across SCF survey years. It also compares those estimates to means computed directly from the Federal Reserve-provided `nwcat` variable. This separates percentile-category assignment from upstream differences in public-use data values, source precision, or official-public processing.

[Insert Table 1 here]

The revised stack implementation reproduces the Federal Reserve percentile-category assignment procedure in the public-use data. In years with small residual differences from published values, the same discrepancies appear when using the Fed-provided `nwcat` variable directly. This indicates that the remaining differences do not arise from `scf_pctile_sum()`'s percentile-grouping algorithm. They more likely reflect upstream differences in public-use data values, source precision, rounding, or official-public processing.

4 Summary

This technical note discusses the task of summarizing statistics by percentile-defined groups in multiply imputed *Survey of Consumer Finances* microdata. It describes three approaches: a commonsense threshold method, implicate-wise estimation, and the Federal Reserve's official stacking method.

The function `scf_pctile_sum()` provides two clearly distinguished workflows: an implicate-wise, imputation-aware approach for ordinary analysis, and a stacked approach for reproducing Federal Reserve percentile-group point estimates. Tests show that the stacked method reproduces the Federal Reserve percentile-category assignment procedure in the public-use data and closely approximates published top-decile mean net worth estimates across survey years.

- Cohen, Joseph Nathan. 2026a. “Analyzing the Survey of Consumer Finances with scf.” https://academicworks.cuny.edu/qc_pubs/695/.
- Cohen, Joseph Nathan. 2026b. *scf: Analyzing the Survey of Consumer Finances*.
- Little, Roderick J. A., and Donald B. Rubin. 1987. *Statistical Analysis with Missing Data*. Vol. 793. John Wiley & Sons.
- Lumley, Thomas, Peter Gao, Ben Schneider, and Stas Kolenikov. 2026. *survey: Analysis of Complex Survey Samples*.
- Rubin, Donald B. 1987. *Multiple Imputation for Nonresponse in Surveys*. John Wiley & Sons.
- U.S. Federal Reserve Bank. 2023a. “Estimates in Nominal Dollars (3 MB Excel).” https://www.federalreserve.gov/econres/files/scf2022_tables_public_nominal_historical.xlsx.
- U.S. Federal Reserve Bank. 2023b. “Survey of Consumer Finances.” <https://doi.org/10.17016/8799>.

Table 1: Comparison of Federal Reserve Published Estimates, scf Percentile-Group Estimates, and Fed-Provided nwcat Estimates for Mean Net Worth Among the Top 10 Percent of Households

Year	Official	'scf_pctile_sum()'	'nwcat'	'scf' difference	'nwcat' difference
1989	\$2,926.55K	\$2,926.55K	\$2,926.55K	\$0.00K	\$0.00K
1992	\$2,595.59K	\$2,595.59K	\$2,595.59K	\$0.00K	\$0.00K
1995	\$2,801.32K	\$2,801.32K	\$2,801.32K	\$0.00K	\$0.00K
1998	\$3,545.41K	\$3,545.41K	\$3,545.41K	\$0.00K	\$0.00K
2001	\$4,629.55K	\$4,629.55K	\$4,629.55K	\$0.00K	\$0.00K
2004	\$4,900.37K	\$4,900.29K	\$4,900.29K	\$0.08K	\$0.08K
2007	\$5,688.07K	\$5,687.94K	\$5,687.94K	\$0.13K	\$0.13K
2010	\$5,028.69K	\$5,026.44K	\$5,026.44K	\$2.25K	\$2.25K
2013	\$5,043.52K	\$5,043.50K	\$5,043.50K	\$0.02K	\$0.02K
2016	\$6,550.28K	\$6,550.24K	\$6,550.24K	\$0.04K	\$0.04K
2019	\$6,619.46K	\$6,619.35K	\$6,619.35K	\$0.11K	\$0.11K
2022	\$7,771.29K	\$7,771.16K	\$7,771.16K	\$0.13K	\$0.13K